Assignment 1 - Transform Hierarchy for an Animated Character

The purpose of this assignment is to use a hierarchy of transformations to make an articulated character. You must develop a hierarchy which is useful for posing the character, and ultimately create a short keyframe animation. Through this assignment you will develop an understanding of canonical transformations and put into practice composition of transformations to produce a desired result.

Getting Started

It is recommended that you install Anaconda3. You should use a *clean* environment to avoid problems! Use python 3.11 for best compatibility for all assignments this term. Create and activate an environment specific to this course with the following commands:

```
conda create --name comp557f26 python=3.11
conda activate comp557f26
```

You must tell VS Code (or your IDE of preference) to use this environment. In VS Code, press Ctrl+Shift+P, then type "Python: Select Interpreter", then find the environment you created in the list.

There are several requirements to install for the assignment:

```
python -m pip install -r requirements.txt
```

See the polyscope example shown in class, and the following module documentation links will likewise be useful to consult:

- numpy note the "absolute beginners guide"
- trimesh, i.e., just the creation methods
- polyscope

You will need to create and compose various canonical transformations in this assignment. You should write your own helper functions to create numpy matrices for these transformations.

Overview

Your character will be a collection of rigid solid objects "connected" by parametric joints, such as single axis rotary joints, or three-axis spherical joints. There is a lot of room for creativity in making a unique character with a given personality or appearance. Have fun! Note that to receive full marks your character should have a minimum level of complexity, i.e., torso, head, two arms, two legs, with elbows and knees, or something of equivalent complexity.

Some code is provided to help you draw the different body parts of your character, but others must be created as objectives. See comments in the provided character.json5 file for hints about building transforms and the hierarchy.

Objectives

1. **Matrix Transformations (1 mark)**
   Create helper functions to create canonical 4x4 affine transformation matrices for translation and rotation.
2. **Compose Transformations for Pose Update (1 mark)**
   Finish implementing the `Node.pose_update` method so that the correct transformation is computed for the node based on the current values of the parameters. The `pose_update` method should recursively compute the transformation from local to world coordinates, and update the polyscope transform for the node's geometry. You will need to compose the transformations in the order specified in the `dofs` property of your character's JSON5 file. Use the helper functions you implemented in the previous objective to create the translation and rotation matrices.
3. **Add Degrees of Freedom (1 mark)**
   Add the ability to control the degrees of freedom for each node in your character. The `dofs` property of your character's JSON5 file specifies the order of the transformations to apply to the node. For instance, a node with `dofs="rz ry rx"` will have three rotation parameters, and the transformations should be constructed with matrix multiplication in this order (i.e., geometry is transformed first by rx, then ry, then rz).
   You should also support "tx", "ty", and "tz" DOFs for translation, and use these for the root node of your character (translation could also be useful for telescopic arms or legs).
   The provided code automatically exposes these parameters in the user interface.
4. **Colour changes (1 mark)**
   Add support for colour changes, e.g., "cr cg cb ca" for the red, green, blue, and transparency parameters to the DOFs. Here "ca" represents colour alpha, for transparency (your character can turn invisible?). Update the node's polyscope `ps_mesh` accordingly.
5. **Other Geometry Types (1 mark)**

The provided code only supports capsules defined in the JSON5 file. Update the code to support at least 2 other geometry types (e.g., icosphere, cone, box), and be sure to use these geometry types in your character. Note how the provided code passes parameters you list in the `character.json5` file directly to the geometry creation functions.

6. **Create a Character (2 marks)**
   While you will likely have already expanded the provided JSON5 file (or have made new files) to test your code so far, this objective asks you to design a character of sufficient complexity as described earlier. Disable the axis display once you are comfortable with your character design. You will submit your character's JSON5 file along with your code.
7. **Sample Pose (1 mark)**
   Having finished creating your character, create a pose to show your character to the world. Take a screen capture and save to an image file (`png` or `jpg`) and submit this along with your other materials. Your filename should have the form `lastname-description.png` where `lastname` is your last name and `description` is the name or description of your character.
8. **Keyframe Animation (1 mark)**
   You must create a simple keyframe animation to bring your character to life. Perhaps have your character wave hello, or try to communicate with gestures, or something more difficult such as a walk cycle! Note that choices you make in the parameters of joints might make some poses challenging (i.e., gimbal lock).
   Simple keyframing code is already written for you. Use the controls to navigate between keyframes, and to create, update, or delete keyframes.
   One strategy for creating an animation is to create a number of keyframes with the basic motion of just a few joints, and then revisit each frame making additional adjustments and updating the keyframes with the new poses. Feel free to add features to simplify the process (e.g., copy and paste).
9. **Readme File (1 mark)**
   Update the provided `README.md` with answers to questions in the file. Also, use the file to briefly explain, in a maximum of 1000 characters, decisions you made in writing the code, creating your character or the animation.

Finished?

Great! Submit to MyCourses your python implementation, data files, pose image, and readme. Use a `zip` archive (only use `zip` format). Please do not submit any unnecessary files (e.g., *_pycache_* folder, .vscode folder, this specification, etc.).
Use the comments at the top of the files you submit to list your name and student number.

DOUBLE CHECK your submitted files by downloading them. You will not receive marks for assignments if files are missing, if it is a different assignment, or if the files are corrupt!

Recall the readme can be used to request a late penalty waiver following the mechanism described in the first lecture slides.

Note that you are encouraged to discuss assignments with your classmates, but not to the point of sharing code and answers. All code and written answers must be your own. Please see the course outline and the fine print in the slides of the first lecture.